



Tecnológico de Monterrey

Evidencia 1 - Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales

**Programación de estructuras de datos y algoritmos fundamentales
| TC1031.607**

David Alonso Cantú Delgado.

Regina Mugica Limon | A00843542

*20 de septiembre del 2026
Monterrey, Nuevo León*

Plan de pruebas

Para comprobar el funcionamiento del programa realicé diferentes pruebas utilizando los dos archivos proporcionados. Probé los siete algoritmos de ordenamiento con cada archivo para comparar sus tiempos de ejecución. También realicé búsquedas por rango de fechas para comprobar el funcionamiento de la búsqueda binaria y probé casos especiales como rangos sin resultados, formatos inválidos y registros con fechas y horas repetidas.

Prueba	Qué se quiere comprobar
7 algoritmos con <code>log607-1.txt</code>	Funcionamiento con datos desordenados
7 algoritmos con <code>log607-2.txt</code>	Funcionamiento con datos casi ordenados
Rango normal	Que encuentre todos los registros dentro del rango
Rango vacío	Que indique que no existen registros
Formato inválido	Que detecte una fecha incorrecta
Timestamp duplicado	Que incluya todos los registros con la misma fecha y hora

Resultados de los algoritmos

Se ejecutaron los siete algoritmos de ordenamiento con los dos archivos proporcionados. Ambos archivos contienen 6,818 registros, pero `log607-1.txt` se encuentra desordenado, mientras que `log607-2.txt` está casi ordenado. Para cada ejecución se realizó una predicción antes de conocer el tiempo y posteriormente se comparó con el resultado obtenido.

Algoritmo	<code>log607-1.txt</code> (desordenado)	<code>log607-2.txt</code> (casi ordenado)
Swap Sort	1,380,312,959 ns	662,691,709 ns
Bubble Sort	1,432,364,292 ns	10,759,542 ns
Selection Sort	679,058,125 ns	665,145,333 ns
Insertion Sort	862,622,833 ns	5,362,208 ns
Merge Sort	35,419,917 ns	31,734,459 ns
Quick Sort	14,111,625 ns	212,582,959 ns
Shell Sort	25,842,042 ns	14,374,750 ns

Los resultados muestran que el orden inicial de los datos afecta de manera diferente a cada algoritmo. Bubble Sort e Insertion Sort fueron mucho más rápidos con el archivo casi ordenado. Selection Sort presentó tiempos muy similares en ambos archivos, ya que sigue recorriendo los elementos para encontrar el menor aunque los datos ya estén casi ordenados. Merge Sort también presentó tiempos similares debido a que mantiene una complejidad de $O(n \log n)$.

Un resultado que llamó mi atención fue Quick Sort. En el archivo desordenado tardó 14,111,625 ns, mientras que en el archivo casi ordenado tardó 212,582,959 ns. Esto ocurrió porque en mi implementación se utiliza el último elemento como pivote, por lo que un archivo casi ordenado puede generar particiones menos balanceadas y acercar al algoritmo a su peor caso $O(n^2)$. En esta prueba, mi predicción de que podría ser más lento sí coincidió con el resultado obtenido.

Casos de prueba

Prueba 1: Búsqueda por rango normal. Se comprobó que el programa encontrara correctamente los registros dentro de un rango de fechas.

```

----- RESULTADOS -----
Algoritmo: Merge Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Tiempo de ejecución: 30196125 nanosegundos
Mejor caso:  $O(n \log n)$ 
Peor caso:  $O(n \log n)$ 
Prediccion inicial: Espero que sea rapido porque Merge Sort tiene complejidad  $O(n \log n)$ .
Coincidio con la prediccion: si

Primer registro despues de ordenar:
Sep 8 2024 00:22:43

Archivo output607.txt creado correctamente.

Ingresa la fecha y hora inicial:
Formato: Mes Dia Anio HH:MM:SS
Sep 08 2024 00:00:00

Ingresa la fecha y hora final:
Formato: Mes Dia Anio HH:MM:SS
Sep 08 2024 23:59:59

Posicion inicial: 0
Posicion final: 7
Registros encontrados: 8
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
Sep 08 2024 10:42:42 10.14.40.195 Data corruption
Sep 08 2024 10:47:52 10.14.24.223 Data corruption
Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
Archivo range607.txt creado correctamente.

```

Prueba 2: rango vacío. Se comprobó el comportamiento del programa cuando no existen registros dentro del rango.

```

----- RESULTADOS -----
Algoritmo: Merge Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Tiempo de ejecucion: 32506417 nanosegundos
Mejor caso:  $O(n \log n)$ 
Peor caso:  $O(n \log n)$ 
Prediccion inicial: Espero que sea rapido porque Merge Sort tiene complejidad  $O(n \log n)$ .
Coincidio con la prediccion: si

Primer registro despues de ordenar:
Sep 8 2024 00:22:43

Archivo output607.txt creado correctamente.

Ingresa la fecha y hora inicial:
Formato: Mes Dia Anio HH:MM:SS
Jan 01 2020 00:00:00

Ingresa la fecha y hora final:
Formato: Mes Dia Anio HH:MM:SS
Jan 02 2020 23:59:59

No se encontraron registros en ese rango.

```

Prueba 3: formato inválido. Se comprobó que el programa detectara una fecha con formato inválido.

```

----- RESULTADOS -----
Algoritmo: Merge Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Tiempo de ejecucion: 32773375 nanosegundos
Mejor caso:  $O(n \log n)$ 
Peor caso:  $O(n \log n)$ 
Prediccion inicial: Espero que sea rapido porque Merge Sort tiene complejidad  $O(n \log n)$ .
Coincidio con la prediccion: no

Primer registro despues de ordenar:
Sep 8 2024 00:22:43

Archivo output607.txt creado correctamente.

Ingresa la fecha y hora inicial:
Formato: Mes Dia Anio HH:MM:SS
Hola 45 2024 20:00:00

Ingresa la fecha y hora final:
Formato: Mes Dia Anio HH:MM:SS
Oct 02 2024 23:00:00

Formato de fecha invalido.

Deseas realizar otra prueba?
1. Si
2. No
Opcion: █

```

Prueba 4: timestamp duplicado. Se comprobó que la búsqueda incluyera los dos registros con el mismo timestamp.

```

----- RESULTADOS -----
Algoritmo: Merge Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Tiempo de ejecución: 32727583 nanosegundos
Mejor caso:  $O(n \log n)$ 
Peor caso:  $O(n \log n)$ 
Predicción inicial: Espero que sea rapido porque Merge Sort tiene complejidad  $O(n \log n)$ .
Coincidio con la predicción: si

Primer registro despues de ordenar:
Sep 8 2024 00:22:43

Archivo output607.txt creado correctamente.

Ingresa la fecha y hora inicial:
Formato: Mes Dia Anio HH:MM:SS
Oct 02 2024 20:00:00

Ingresa la fecha y hora final:
Formato: Mes Dia Anio HH:MM:SS
Oct 02 2024 23:04:24

Posicion inicial: 219
Posicion final: 221
Registros encontrados: 3
Oct 02 2024 21:51:03 10.14.193.75 Anomalous User Activity
Oct 02 2024 23:04:24 10.14.121.223 Changes in File Integrity
Oct 02 2024 23:04:24 10.14.47.219 Social engineering attacks
Archivo range607.txt creado correctamente.

```

Prueba 5. Límite exacto. Se comprobó que un registro que coincide exactamente con los límites de búsqueda fuera incluido.

```

----- RESULTADOS -----
Algoritmo: Merge Sort
Archivo: log607-2.txt
Cantidad de registros: 6818
Tiempo de ejecución: 33278459 nanosegundos
Mejor caso:  $O(n \log n)$ 
Peor caso:  $O(n \log n)$ 
Predicción inicial: Espero que sea rapido porque Merge Sort tiene complejidad  $O(n \log n)$ .
Coincidio con la predicción: si

Primer registro despues de ordenar:
Sep 8 2024 00:22:43

Archivo output607.txt creado correctamente.

Ingresa la fecha y hora inicial:
Formato: Mes Dia Anio HH:MM:SS
Sep 08 2024 00:22:43

Ingresa la fecha y hora final:
Formato: Mes Dia Anio HH:MM:SS
Sep 08 2024 00:22:43

Posicion inicial: 0
Posicion final: 0
Registros encontrados: 1
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Archivo range607.txt creado correctamente.

```

Conclusión

A partir de las pruebas realizadas pude observar que el orden inicial de los datos sí puede afectar de manera importante el rendimiento de algunos algoritmos. Bubble Sort e Insertion Sort fueron mucho más rápidos con el archivo casi ordenado, mientras que Selection Sort tuvo tiempos muy similares en ambos archivos. Merge Sort también mantuvo tiempos parecidos debido a su complejidad $O(n \log n)$.

El caso que más me llamó la atención fue Quick Sort, ya que fue muy rápido con el archivo desordenado, pero mucho más lento con el archivo casi ordenado. Esto se relaciona con que mi implementación utiliza el último elemento como pivote y puede generar particiones poco balanceadas cuando los datos ya tienen cierto orden.

Las pruebas de búsqueda también permitieron comprobar que el programa encuentra correctamente los registros dentro de un rango, incluye los registros que coinciden exactamente con los límites y considera timestamps duplicados. Además, responde correctamente ante rangos sin resultados y entradas inválidas.